Что такое Git и GitLab?

Git - это системы контроля версий (Version Control System).

Представьте, что вы пишете документ:

1. Вы бы сделали document_v1.txt document_v2.txt document_final document_real_final.txt

2. С Git вам даже не нужно переименовывать файлы. Git сам запомнит, что вы изменили, кто и когда сделал изменения.

Аналогия: Git - это как "машина времени" для ваших файлов, которая запоминает каждое изменение.

Что такие GitLab?

GitLab - это веб-платформа для работы с Git. Если Git - это двигатель, то GitLab - это автомобиль, с удобным рулем и педалями.

Основные возможности GitLab:

1. Хранилище кода (как Google Drive для программистов)
2. Отслеживание изменений (история)
3. Отслеживание ошибок (issues)
4. Непрерывная интеграция (CI/CD)
5. Code review

с чего мы начинаем ?

глобальная установка git на ноутбук/компьютер

https://git-scm.com/downloads#win

после установки проверяем версию

git --version

первоначальная настройка в терминале (ВАЖНО !!!)

git config --global name"**Ваше Имя**"

git config --global email"**ваш.email@example.com**"

дополнительные настройки

Установка основного редактора (по умолчанию)
git config --global editor code --wait" # для VS Code

или для простого текстового редактора
git config --global editor nano"

Цветной вывод
git config --global ui auto

Проверка настроек
git config --list

Основные концепции Git

1. Репозиторий (Repository)

1. Это папка, за которой следит Git
2. Содержит все файлы проекта и историю изменений

Коммит (Commit)

1. "Снимок" ваших файлов в определенный момент времени
2. Как сохранение в видеоигре или "точка на карте"
3. Содержит сообщение о том, что было изменено

3. Ветка (Branch)

1. Параллельная версия проекта
2. Позволяет работать над разными функциями одновременно

Рабочая область (Working Directory)

1. Файлы, с которыми вы работаете прямо сейчас

Базовые команды Git

Создание репозитория

Способ 1: Создать новый репозиторий
mkdir my-project
cd my-project
git init

Способ 2: Клонировать существующий репозиторий

git clone https://gitlab.com/username/project-name.git

Проверка статуса

git status
Показывает: какие файлы изменены, какие готовы к коммиту

Добавление файлов в отслеживание

Добавить конкретный файл
git add filename.txt

Добавить все файлы в текущей папке
git add .
(точкаа важна!)

Добавить все файлы в проекте
git add -all

Создание коммита

git commit -m "Ваше сообщение о изменениях"

Правила хорошего сообщения коммита

1. Краткое описание (до 50 символов)
2. Более подробное описание (если нужно)
3. Например: "Исправлена ошибка в расчете цены"

Отправить изменения

git push origin ваша ветка

дополнительные изменения (ВАЖНО!)

подтягиваем изменения из основной ветки (нужно делать из этой же ветки)

git pull origin develop

Скачивает изменения из удаленного репозитория, но не применяет их к вашей рабочей ветке

git fetch

создание новой ветки
git branch name ветки

git branch - название ветки

переход на существующую ветку
git checkout **название ветки**

создание и сразу переход на созданную ветку

git checkout -b **название ветки**

посмотреть какие есть ветки **локально**

git branch

получение изменений из основной ветки в ветку где мы работает

1. переходим из нашей рабочей ветки в основную (обычно develop)

git checkout develop

2. подтягиваем обновление develop которые внесли другие разработчики

git pull origin develop

3.откуда у нас актуальная ветка develop нам нужно он вместе в свою ветку (если ветка будет отставать то при вливании ветки в gl он не даст влить же develop т.е ветка будет отставать и нам будут конфликты)

переходим в нашу рабочую ветку

git checkout название ветки

вливаем обновления из ветки develop в нашу ветку

git merge develop

после ветки не сильно отстает и нет конфликтов то ветки обновиться и можно продолжить работать или запомнить свой работу в gitlab , и если есть конфликты то среда разработки (IDE) покажет в каких файлах есть конфликты и нужно вручную выбрать что оставить или отменить изменения или сохранить или продолжить работать(рекомендуется сделать и обновляться каждый раз перед сдачей работы)

ssh

при первом подключении к закрытому репозиторию нужно ключ ssh

Генерация SSH ключа (через консоль)

ssh keygen -t rsa

Копирование публичного ключа

cat ~/.ssh/id_ed25519.pub

Добавление ключа в GitLab:

Settings → SSH Keys → Добавить ключ

.gitignore

- Логи
- *.log

- Зависимости
node_modules/
vendor/

- Файлы IDE
.vscode/
.idea/
*.swp

- Системные файлы
DS_Store
Thumbs.db

- Конфиденциальные данные
.env
config/secrets.yml

Отмена изменений

- Отмена последнего коммита (сохранение изменения)

git reset --softHEAD~1

- Полная отмена последнего коммита

git reset --hardHEAD~1

- Отмена конкретного коммита

git revert commit-hash